APPANROM: A TANGLISH PROGRAMMING LANGUAGE AND ANDROID APP DEVELOPMENT STUDIO

A PROJECT REPORT

Submitted By

P. SANTHOSH	**310124104083**
SHALOM ARPUTHA SINGH	**310124104089**

In Partial Fulfillment for The Award of The Degree

Of

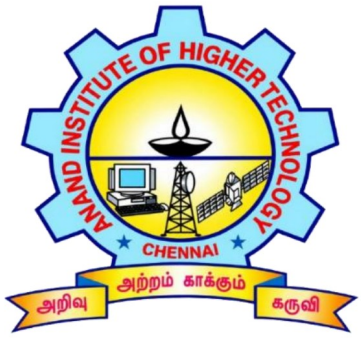
BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

ANAND INSTITUTE OF HIGHER TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)



ANNA UNIVERSITY: CHENNAI 600025

NOVEMBER 2026

1. ABSTRACT

AppPanrom is a web-based programming language and development studio created to let people build Android apps by writing in natural Tanglish, which is everyday spoken Tamil mixed with familiar English programming words, written in the Roman alphabet. A program is written with constructs such as `app_uruvakku`, `screen_uruvakku`, `button(..., click_pannumbodhu=handler)`, `mari` for variables, `enna_na / illana` for conditions, `ovvoru` and `irukkura_varai` for loops, functions, and `try / catch` for error handling. The language toolchain is implemented in Python and follows the traditional compiler structure. Lexical analysis converts the indentation-sensitive source text into tokens. A recursive-descent parser checks the grammar and builds an abstract syntax tree with operator precedence. Semantic analysis then validates variable scopes, function arities, event handlers, navigation targets, component arguments, and style values, and reports every problem with a file name, line number, column and a suggestion. Programs that pass are lowered into a stable JSON intermediate representation (IR) that also lists the Android permissions the program needs. A single dependency-free JavaScript runtime interprets this IR under strict step, recursion and size limits, and renders it as an interactive phone screen. The same runtime file is used by the live preview in the Studio and by the WebView inside the generated Android application, so both behave alike. The Studio, written in React and TypeScript with a FastAPI backend, includes a code editor with live diagnostics, a phone-frame preview with a console, and a visual designer in which components are dragged from a palette onto the phone, moved, edited in a property inspector, and deleted; every designer action is turned into a text edit of the source code, and every change typed in the code redraws the canvas, so the two always stay in step. An Android build engine generates a Gradle project from the IR, runs the build in a background job with logging and a time limit, verifies the produced debug APK, release APK or Android App Bundle, handles the app icon and permissions, and keeps signing keys out of source code and logs. The system is covered by 114 backend tests, 30 runtime tests and a browser end-to-end suite. As a result, AppPanrom functions both as a working compiler toolchain and as a practical environment for creating Android apps in a Tanglish language.

2. OBJECTIVES

- To design and implement a multi-phase compiler for a Tanglish language with variables, conditions, loops, functions, events, screens and UI components, producing clear diagnostics with line and column numbers.
- To lower every valid program into one validated intermediate representation that is shown identically in the live preview and in the Android application, using one shared runtime and one shared component and style specification.
- To provide a visual designer with drag and drop and a property inspector, as in Android Studio, whose actions update the source code live, and whose code edits update the design live.
- To automate Android packaging: generate a Gradle project, build a debug or signed release APK or AAB in a controlled background job, verify the artifact, and make it available for download only when it is real.
- To keep the system safe and local-first: no execution of user code as Python or JavaScript, resource limits, validated file paths, and signing secrets that never reach files, logs or the database.

3. PROPOSED ARCHITECTURE

The system has five layers:

- **Presentation layer (frontend/, React and TypeScript):** dashboard, code editor with highlighting, phone-frame preview, console, diagnostics panel, palette, design canvas, outline tree, property inspector and build panel.
- **Language engine (backend/apppanrom/lang):** lexer, parser, semantic analyzer, IR generator, and the designer engine that rewrites source text.
- **Shared runtime (runtime/apppanrom-runtime.js):** controlled interpreter and renderer used by the preview and by the APK.
- **Service layer (FastAPI, SQLite):** projects, files, validation, design edits, build jobs, logs and artifact download.
- **Android build engine:** project generator, Gradle build manager, prerequisite and signing checks, and artifact verification. External tools: Gradle, the Android SDK, keytool and jarsigner.

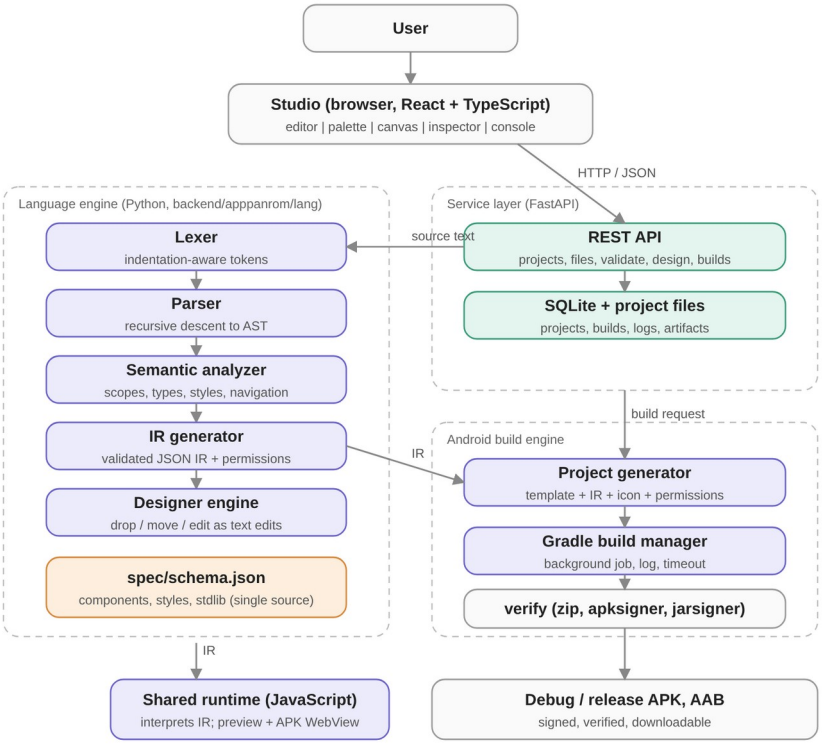


Fig. A: Layered architecture of AppPanrom

4. SYSTEM WORKFLOW

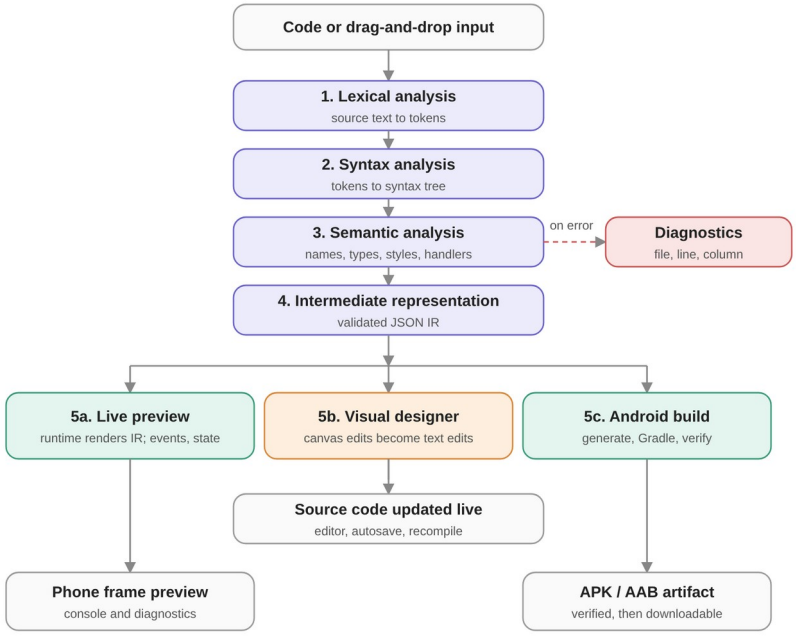


Fig. B: From source or drag-and-drop input to preview, design and Android build

5. ADVANTAGES

- **Complete toolchain in one product:** lexical, syntax and semantic analysis, an intermediate representation, a runtime, a visual designer and an Android build pipeline work together in a single system.
- **Natural Tanglish syntax:** familiar English terms are kept (app, screen, title, button, input) while everyday Tamil words are used for actions (uruvakku, kaattu, pannu, edu), so the language reads like spoken Tanglish.
- **Preview equals app:** the live preview and the generated APK run the same runtime file on the same IR, so what is seen in the Studio is what the phone shows.
- **Live two-way design:** dragging, dropping and inspector edits are written into the code, and code edits redraw the canvas, so beginners and programmers can work in the same file.
- **Helpful errors:** every problem is reported with file, line, column and a suggestion, and invalid designer edits are rejected before they are written.
- **Honest and safe builds:** an APK is offered only after Gradle succeeds and the file is verified; signing keys stay in the environment, an unsigned or debug-signed release is rejected, and permissions are declared only for features actually used.
- **Automatically tested:** 114 backend tests, 30 runtime tests, and a Chromium end-to-end suite that drives the real Studio, including drag and drop.

APPENDIX – I
SOURCE CODE

A.1 The language (a minimal AppPanrom program)

Indentation is four spaces; event handlers are named functions; the screen is a block of components.

```
app_uruvakku("Vanakkam App")
mari clicks = 0
function welcome():
    clicks = clicks + 1
screen_uruvakku("home"):
    button("Click Pannu", click_pannumbodhu=welcome)
```

A.2 Compiler pipeline (backend/apppanrom/lang/compiler.py)

```
    def compile(self, source: str) -> CompileResult:
        try:
            program = parse(source)
        except SyntaxProblem as e:
            return CompileResult(False, [e.diagnostic])

        for st in program:  # pass 1: declarations
            t = st["t"]
            if t == "function":
                if st["n"] in self.functions:
                    self.error(st, f"Function '{st['n']}' is already defined.", "duplicate_function")
                elif st["n"] in self.spec["functions"] or st["n"] in self.spec["components"]:
                    self.error(st, f"'{st['n']}' is a built-in name and cannot be redefined.",
    "reserved_name")
                else:
                    self.functions[st["n"]] = st
            elif t == "block_call" and st["f"] == "screen_uruvakku":
                name = self._screen_name(st)
                if name is not None:
                    if name in self.screens:
                        self.error(st, f"Screen '{name}' is declared more than once.",
    "duplicate_screen")
                    else:
                        self.screens[name] = st

        ir_globals: list[dict] = []
        ir_screens: dict[str, dict] = {}
        for st in program:  # pass 2: bodies
            t = st["t"]
            if t == "call_stmt" and st["f"] in self.spec["config_functions"]:
                self._config(st)
            elif t == "let":
                e = self._expr(st["e"], self.globals, None)
                if st["n"] in self.globals:
                    self.error(st, f"Variable '{st['n']}' is already declared.", "duplicate_variable")
                self.globals.add(st["n"])
        # ... pass 2 (bodies), screens and app metadata: see compiler.py
```

A.3 Lexical analyzer (backend/apppanrom/lang/lexer.py)

```
def tokenize(source: str) -> list[Token]:
    if len(source.encode("utf-8")) > MAX_SOURCE_BYTES:
        raise SyntaxProblem(f"Source is larger than {MAX_SOURCE_BYTES} bytes.", 1, 1,
    "source_too_large")
    tokens: list[Token] = []
    indents = [0]
    depth = 0  # open brackets: newlines are ignored inside them
```

```
    lines = source.replace("\r\n", "\n").replace("\r", "\n").split("\n")
    for lineno, raw in enumerate(lines, start=1):
        i = 0
        n = len(raw)
        if depth == 0:
            stripped = raw.lstrip(" ")
            if "\t" in raw[: n - len(raw.lstrip())]:
                raise SyntaxProblem("Tabs are not allowed for indentation.", lineno, 1, "tab_indent",
                                    "Use 4 spaces per indentation level.")
            if stripped == "" or stripped.startswith("#"):
                continue
            width = n - len(stripped)
            if width % INDENT_WIDTH:
                raise SyntaxProblem(f"Indentation must be a multiple of {INDENT_WIDTH} spaces (found
    {width}).",
                                    lineno, 1, "bad_indent", f"Use {INDENT_WIDTH} spaces per level.")
            if width > indents[-1]:
                if width != indents[-1] + INDENT_WIDTH:
                    raise SyntaxProblem("Indented too far; indent one level (4 spaces) at a time.",
    lineno, 1, "bad_indent")
                indents.append(width)
                tokens.append(Token("INDENT", None, lineno, 1))
            else:
                while width < indents[-1]:
                    indents.pop()
                    tokens.append(Token("DEDENT", None, lineno, 1))
                if width != indents[-1]:
                    raise SyntaxProblem("Unindent does not match any outer indentation level.", lineno,
    1, "bad_indent")
            i = width
        while i < n:
            ch = raw[i]
            col = i + 1
            if ch in " \t":
                i += 1
    # ... numbers, names, operators and brackets: see lexer.py
```

A.4 Syntax analyzer (backend/apppanrom/lang/parser.py)

```
    def or_expr(self):
        left = self.and_expr()
        while self.is_kw("or"):
            t = self.advance()
            left = {"t": "bin", "op": "or", "l": left, "r": self.and_expr(), "pos": self.pos(t)}
        return left

    def compare(self):
        left = self.additive()
        if self.cur.kind == "OP" and self.cur.value in COMPARE_OPS:
            t = self.advance()
            left = {"t": "bin", "op": t.value, "l": left, "r": self.additive(), "pos": self.pos(t)}
            if self.cur.kind == "OP" and self.cur.value in COMPARE_OPS:
                self.err("Chained comparisons are not supported.", suggestion="Use 'and': a < b and b <
    c")
        return left

    def additive(self):
        left = self.term()
        while self.cur.kind == "OP" and self.cur.value in "+-":
            t = self.advance()
            left = {"t": "bin", "op": t.value, "l": left, "r": self.term(), "pos": self.pos(t)}
        return left
```

A.5 Semantic analyzer (backend/apppanrom/lang/compiler.py)

```
    def _handler(self, node: dict, event: str) -> str:
        if node["t"] != "name":
            self.error(node, f"{event} must be the name of a function, e.g. {event}=welcome (no quotes,
    no parentheses).", "bad_handler")
            return ""
        fn = self.functions.get(node["n"])
        if fn is None:
            self.error(node, f"Handler function '{node['n']}' is not defined.", "undefined_handler",
                       self.suggest(node["n"], self.functions) or f"Define it: function {node['n']}():")
        elif fn["params"]:
            self.error(node, f"Handler '{node['n']}' must take no parameters.", "handler_params")
        return node["n"]

    def _style_value(self, key: str, d: dict, v: dict):
        lt = literal_type(v)
        if lt is None:
            return  # dynamic value: validated again by the runtime
        typ = d["type"]
        ok = True
        if typ == "color":
            ok = lt == "str" and bool(HEX_RE.fullmatch(v["v"]))
            hint = 'Use a hex color like "#2563EB" or "#FFF".'
        elif typ == "number":
            ok = lt == "num" and d["min"] <= v["v"] <= d["max"]
            hint = f"Use a number from {d['min']} to {d['max']}."
        elif typ == "size":
            ok = (lt == "num" and 0 <= v["v"] <= 2000) or (lt == "str" and
    bool(SIZE_RE.fullmatch(v["v"])))
            hint = 'Use a number, a percentage like "100%", or "fill".'
        elif typ == "enum":
            ok = lt == "str" and v["v"] in d["values"]
            hint = f"Allowed: {', '.join(d['values'])}."
        else:  # bool
            ok = lt == "bool"
            hint = "Use true or false."
        if not ok:
            self.error(v, f"Invalid value for style '{key}'.", "bad_style_value", hint)
```

A.6 Runtime interpreter (runtime/apppanrom-runtime.js)

The interpreter walks the validated IR; there is no eval or exec, and a step limit stops endless loops.

```
    function evalBin(e, env) {
      var op = e.op;
      if (op === 'and') { var l = evalExpr(e.l, env); return truthy(l) ? truthy(evalExpr(e.r, env)) :
    false; }
      if (op === 'or') { var l2 = evalExpr(e.l, env); return truthy(l2) ? true : truthy(evalExpr(e.r,
    env)); }
      var a = evalExpr(e.l, env), b = evalExpr(e.r, env);
      if (op === '==') return deepEqual(a, b);
      if (op === '!=') return !deepEqual(a, b);
      var ta = typeOf(a), tb = typeOf(b);
      if (op === '+') {
        if (ta === 'number' && tb === 'number') return finite(a + b, e);
        if (ta === 'text' && tb === 'text') {
          if (a.length + b.length > LIMITS.stringLength) throw new ApError('Text is too long.', e.pos);
          return a + b;
        }
        if (ta === 'list' && tb === 'list') return a.concat(b);
        throw new ApError("Cannot use '+' between " + ta + ' and ' + tb + '.' + (ta === 'text' || tb ===
    'text' ? ' Use str(...) to convert numbers to text.' : ''), e.pos);
```

```
      }
      if (op === '<' || op === '>' || op === '<=' || op === '>=') {
        if (!((ta === 'number' && tb === 'number') || (ta === 'text' && tb === 'text'))) throw new
    ApError("Cannot compare " + ta + " with " + tb + " using '" + op + "'.", e.pos);
        return op === '<' ? a < b : op === '>' ? a > b : op === '<=' ? a <= b : a >= b;
      }
      if (ta !== 'number' || tb !== 'number') throw new ApError("Cannot use '" + op + "' between " + ta
    + ' and ' + tb + '.', e.pos);
      if (op === '-') return finite(a - b, e);
      if (op === '*') return finite(a * b, e);
      if (b === 0) throw new ApError('Division by zero.', e.pos);
      return op === '/' ? finite(a / b, e) : a % b;
    }

    function tick(pos) {
      if (++steps > LIMITS.steps) throw new ApError('Program ran too long (more than ' + LIMITS.steps +
    ' steps); it may contain an endless loop.', pos, true);
    }
    function callUser(name, fn, args, pos) {
      if (args.length !== fn.params.length) throw new ApError("Function '" + name + "' expects " +
    fn.params.length + ' argument(s).', pos);
      if (++depth > LIMITS.depth) throw new ApError('Too much recursion (more than ' + LIMITS.depth + '
    nested calls).', pos, true);
      var env = new Map();
      fn.params.forEach(function (p, i) { env.set(p, args[i]); });
      try {
        var r = execBlock(fn.body, env, null);
        return r ? r.v : null;
      } finally { depth--; }
    }
```

A.7 Design canvas drop target (runtime/apppanrom-runtime.js)

```
    function zoneFor(ev) {
      var t = ev.target && ev.target.closest ? ev.target.closest('[data-ap-line]') : null;
      if (t && !app.contains(t)) t = null;
      if (dragEl && t && (t === dragEl || dragEl.contains(t))) return null;
      if (!t) return { el: null, position: 'inside' };
      var r = t.getBoundingClientRect(), par = t.parentElement;
      var row = !!par && /^row/.test(doc.defaultView.getComputedStyle(par).flexDirection);
      var ratio = row ? (ev.clientX - r.left) / (r.width || 1) : (ev.clientY - r.top) / (r.height || 1);
      if (t.hasAttribute('data-ap-container')) return { el: t, position: ratio < 0.25 ? 'before' : ratio
    > 0.75 ? 'after' : 'inside', row: row };
      return { el: t, position: ratio < 0.5 ? 'before' : 'after', row: row };
    }
```

A.8 Designer engine (backend/apppanrom/lang/designer.py)

Each drag-and-drop action becomes a minimal text edit of the source; the result is parsed again before it is accepted.

```
def _insertion_point(program, target: Loc | None, screen: Loc | None, position: str, edits: Edits) ->
    tuple[int, int]:
    """Returns (0-based index to insert at, indent). May schedule removal of a lone `summa`
    placeholder."""
    if position == "inside":
        box = target or screen
        if box is None or not is_container(box.node):
            raise DesignError("You can only drop inside a container or a screen.", "not_a_container")
        kids = box.node["children"]
        indent = _indent_of(kids[0]) if kids else _indent_of(box.node) + INDENT
        if len(kids) == 1 and kids[0]["t"] == "summa":
```

```
            edits.remove(kids[0]["pos"][0] - 1, kids[0]["end"])
            return kids[0]["pos"][0] - 1, indent
        return box.node["end"], indent
    if target is None:
        raise DesignError("A target component is required.", "bad_target")
    if target.parent is None or not _in_screen(target) or is_screen(target.node):
        raise DesignError("You can only drop next to components inside a screen.", "bad_target")
    if position == "before":
        return target.node["pos"][0] - 1, _indent_of(target.node)
    if position == "after":
        return target.node["end"], _indent_of(target.node)
    raise DesignError("Position must be before, after or inside.", "bad_position")

def _op_move(program, lines, source, op):
    node_loc = _need_component(program, int(op["line"]))
    if is_screen(node_loc.node):
        raise DesignError("Screens cannot be moved here.", "protected")
    a, b = node_loc.node["pos"][0] - 1, node_loc.node["end"]
    screen = None if op.get("target_line") is not None else _first_screen(program, op.get("screen"))
    target = _target(program, op)
    if target is not None and a <= target.node["pos"][0] - 1 < b:
        raise DesignError("A component cannot be moved into itself.", "bad_target")
    edits = Edits()
    idx, indent = _insertion_point(program, target, screen, op.get("position", "inside"), edits)
    if a <= idx < b and idx != a:
        raise DesignError("A component cannot be moved into itself.", "bad_target")
    block = reindent(lines[a:b], indent - _indent_of(node_loc.node))
    edits.remove(a, b)
    _fill_if_empty(edits, node_loc, (a, b))
    edits.insert(idx, block)
    return edits.apply(lines), idx + edits.shift_before(idx) + 1
```

A.9 Android build engine (backend/apppanrom/android/builder.py)

```
// Build job
    def _run(self, build_id: str, project_id: str, ir: dict, kind: str, icon: Path | None) -> None:
        started = time.monotonic()
        work = self.s.builds_dir / build_id / "work"
        try:
            self.db.execute("UPDATE builds SET status='running', started_at=? WHERE id=?", (_now(),
    build_id))
            pre = check_prerequisites(self.s, use_cache=False)
            if not pre["ok"]:
                text = format_problems(pre)
                self._log(build_id, text)
                self._fail(build_id, started, "prerequisites_missing", "Android build prerequisites are
    missing; see the log.")
                return
            task, rel_out, suffix, is_release = KINDS[kind]
            if is_release:
                sign = check_signing(self.s)
                if not sign["configured"]:
                    self._log(build_id, f"{sign['detail']}\n  fix: {sign['fix']}")
                    self._fail(build_id, started, "signing_not_configured", "Release signing is not
    configured; see the log. No unsigned release is ever produced.")
                    return
            self._log(build_id, "[apppanrom] prerequisites OK; generating Android project")
            try:
                generate_project(ir, work, icon)
            except GenerateError as e:
                self._fail(build_id, started, "invalid_project", str(e))
                return
            code, timed_out = self._gradle(build_id, work, task, is_release)
            if timed_out:
```

```
                self._fail(build_id, started, "timeout", f"Build exceeded the {self.s.build_timeout_s}s
    time limit and was stopped.", code)
                return
            if code != 0:
                self._fail(build_id, started, "gradle_failed", f"Gradle exited with code {code}; see the
    log.", code)
            # ... store the verified artifact, clean the work directory

// Release signature check
    def _verify_release_signature(self, build_id: str, path: Path, kind: str) -> tuple[str | None, str]:
        """Independently checks the signature of a release artifact. Returns (problem, human-readable
    signing status)."""
        if kind == "release_apk":
            tool = find_apksigner(self.s)
            if not tool:
                self._log(build_id, "[apppanrom] apksigner not found in the SDK build-tools; the
    signature could not be independently verified")
                return None, "release key (Gradle signed it; not independently verified: apksigner
    missing)"
            rc, out = _run_tool([tool, "verify", "--print-certs", str(path)])
            self._log(build_id, "[apppanrom] apksigner verify:\n" + out.strip()[:2000])
            if rc != 0:
                return "apksigner could not verify the APK signature.", ""
            if "CN=Android Debug" in out:
                return "The release APK is signed with the Android debug key, not a release key.", ""
            return None, "release key (verified with apksigner)"
        tool = find_jarsigner(self.s)
        if not tool:
            self._log(build_id, "[apppanrom] jarsigner not found; the bundle signature could not be
    independently verified")
            return None, "release key (Gradle signed it; not independently verified: jarsigner missing)"
        rc, out = _run_tool([tool, "-verify", "-certs", str(path)])
        self._log(build_id, "[apppanrom] jarsigner -verify:\n" + out.strip()[:2000])
        if rc != 0 or "jar verified" not in out:
            return "jarsigner could not verify the AAB signature (it may be unsigned).", ""
        if "CN=Android Debug" in out:
            return "The release bundle is signed with the Android debug key, not a release key.", ""
        return None, "release key (verified with jarsigner)"

================================================================================
A.10 Android template (android-template/app/build.gradle, excerpt)
================================================================================

plugins {
    id 'com.android.application'
}

android {
    // Fixed code namespace; the user-facing application id is set below.
    namespace 'com.apppanrom.shell'
    compileSdk 34

    defaultConfig {
        applicationId "__APPLICATION_ID__"
        minSdk 24
        targetSdk 34
        versionCode __VERSION_CODE__
        versionName "__VERSION_NAME__"
    }

    // Release signing material is read from the environment of the Gradle process only (never from
    files in this project).
    signingConfigs {
        release {
            def storeFilePath = System.getenv("APPANROM_SIGN_STORE_FILE")
            if (storeFilePath) {
```

```
                storeFile file(storeFilePath)
                storePassword System.getenv("APPANROM_SIGN_STORE_PASSWORD")
                keyAlias System.getenv("APPANROM_SIGN_KEY_ALIAS")
                keyPassword System.getenv("APPANROM_SIGN_KEY_PASSWORD")
            }
        }
    }

    buildTypes {
        release {
            minifyEnabled false
            // Without the environment above this fails loudly instead of producing an unsigned
    "release".
            signingConfig signingConfigs.release
        }
```

```
================================================================================
A.11 Automated test of the designer (backend/tests/test_designer.py)
================================================================================
```

```
def test_move_out_of_container_reindents():
    out = op(BASE, kind="move", line=7, target_line=9, position="after")
    lines = out["source"].split("\n")
    assert lines[-2] == '    title("Hello")' and lines[out["line"] - 1] == '    title("Hello")'

def test_many_random_edits_keep_source_compilable():
    import random
    rnd = random.Random(7)
    s = BASE
    types = ["title", "text_kaattu", "button", "input_uruvakku", "card_uruvakku", "row_uruvakku",
    "column_uruvakku", "list_kaattu", "divider_kaattu"]
    for _ in range(150):
        from apppanrom.lang.parser import parse
```

```
================================================================================
A.12 Sample run
================================================================================
```

```
// Tokens produced by the lexical analyzer (first 10)
#   TOKEN TYPE   VALUE              LINE
1   NAME         app_uruvakku       1
2   OP           (                  1
3   STRING       Vanakkam App       1
4   OP           )                  1
5   KEYWORD      mari               2
6   NAME         clicks             2
7   OP           =                  2
8   NUMBER       0                  2
9   KEYWORD      function           3
10  NAME         welcome            3

// Intermediate representation (excerpt)
{
 "app": {
  "name": "Vanakkam App",
  "id": "com.apppanrom.myapp",
  "version": "1.0.0",
  "theme": "light",
  "start_screen": "home"
 },
 "globals": [
  {
   "t": "let",
   "n": "clicks",
   "e": {
    "t": "num",
```

```
    "v": 0
   }
  }
 ],
 "screens": {
  "home": {
   "body": [
    {
     "t": "comp",
     "c": "button",
     "props": {
      "label": {
       "t": "str",
       "v": "Click Pannu"
      }
     },
     "events": {
      "click_pannumbodhu": "welcome"
     }
    }
   ]
  }
 },
 "permissions": []
}

// Diagnostics for a program with two mistakes
[
 {
  "severity": "error",
  "code": "unknown_function",
  "message": "Unknown function 'titel'.",
  "line": 2,
  "col": 5,
  "suggestion": "Did you mean 'title'?"
 },
 {
  "severity": "error",
  "code": "undefined_handler",
  "message": "Handler function 'missing' is not defined.",
  "line": 3,
  "col": 36,
  "suggestion": "Define it: function missing():"
 }
]

// Designer: text edits for a drop and a drag
# Before
screen_uruvakku("home"):
    column_uruvakku(style={"gap": 8}):
        title("Hello")
    text_kaattu("Footer")

# Drop a Button into the column (insert, position=inside, target_line=2)
screen_uruvakku("home"):
    column_uruvakku(style={"gap": 8}):
        title("Hello")
        button("Button")
    text_kaattu("Footer")

# Drag the footer text into the column (move, line=4, target_line=2, position=inside)
screen_uruvakku("home"):
    column_uruvakku(style={"gap": 8}):
        title("Hello")
        text_kaattu("Footer")
```

APPENDIX – II
SCREENSHOTS

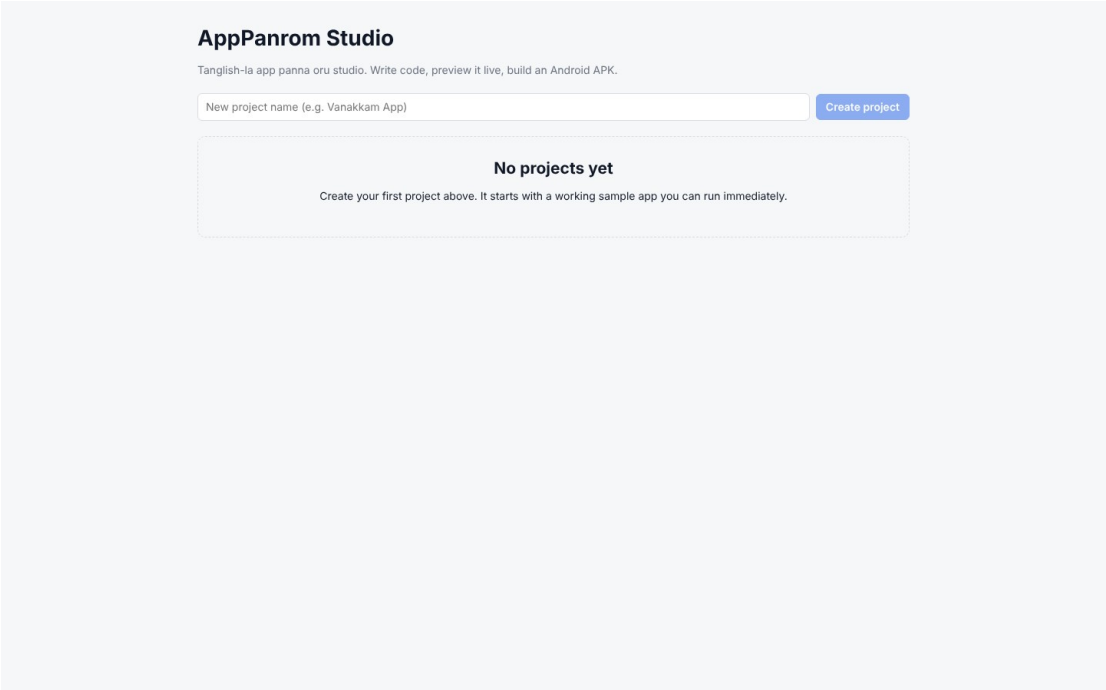


Fig. 1: Project dashboard with the empty state and the new-project form

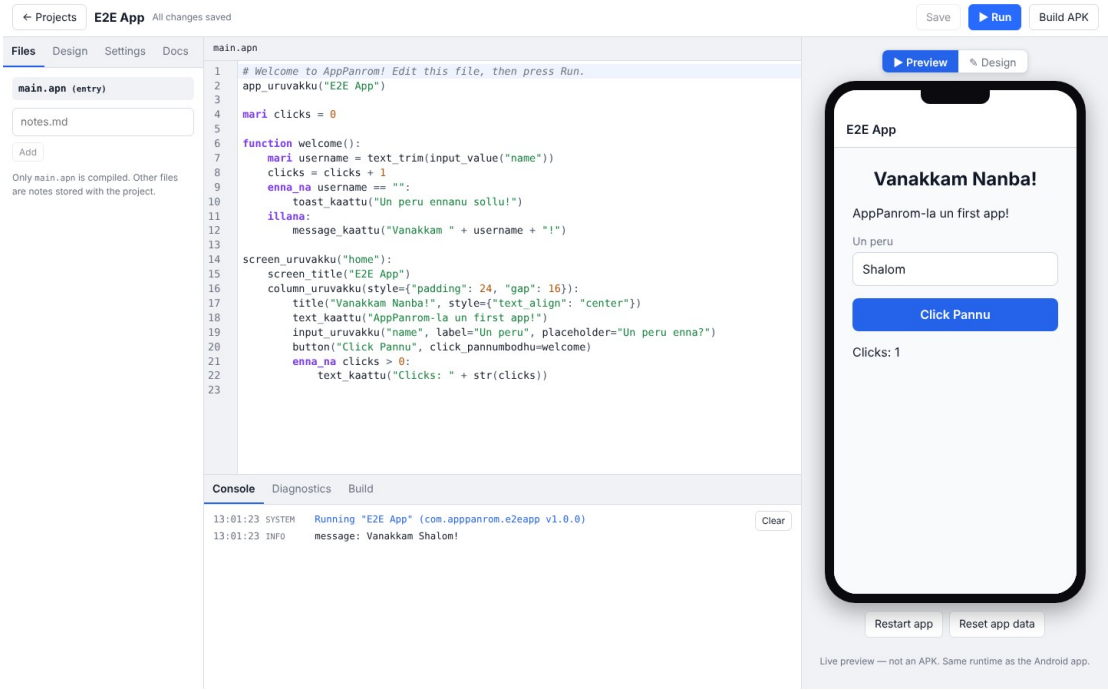


Fig. 2: Code editor with highlighting and the live phone preview after Run

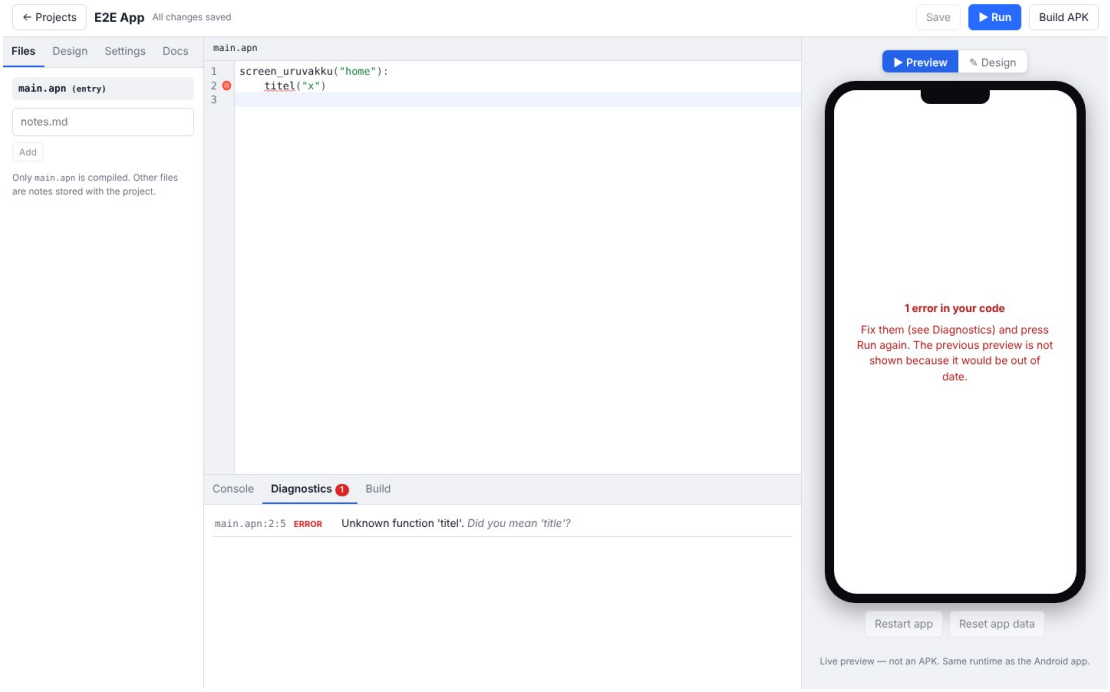


Fig. 3: A mistake in the code: the preview shows an error state and the console reports line and column

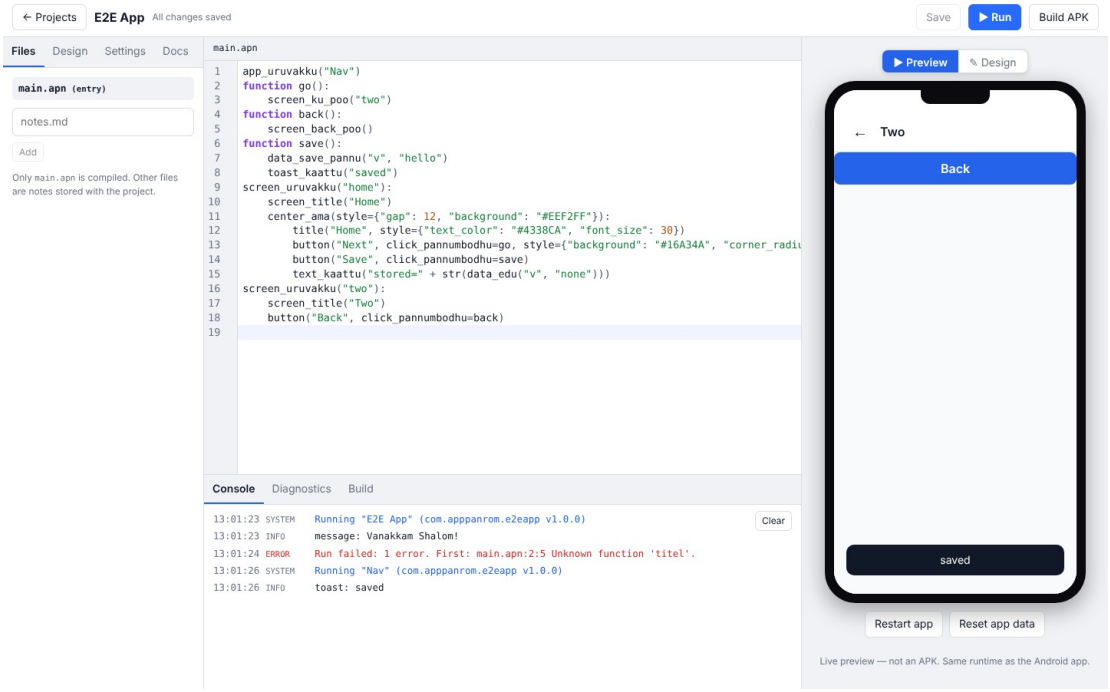


Fig. 4: A styled two-screen app in the preview with navigation, toast and storage

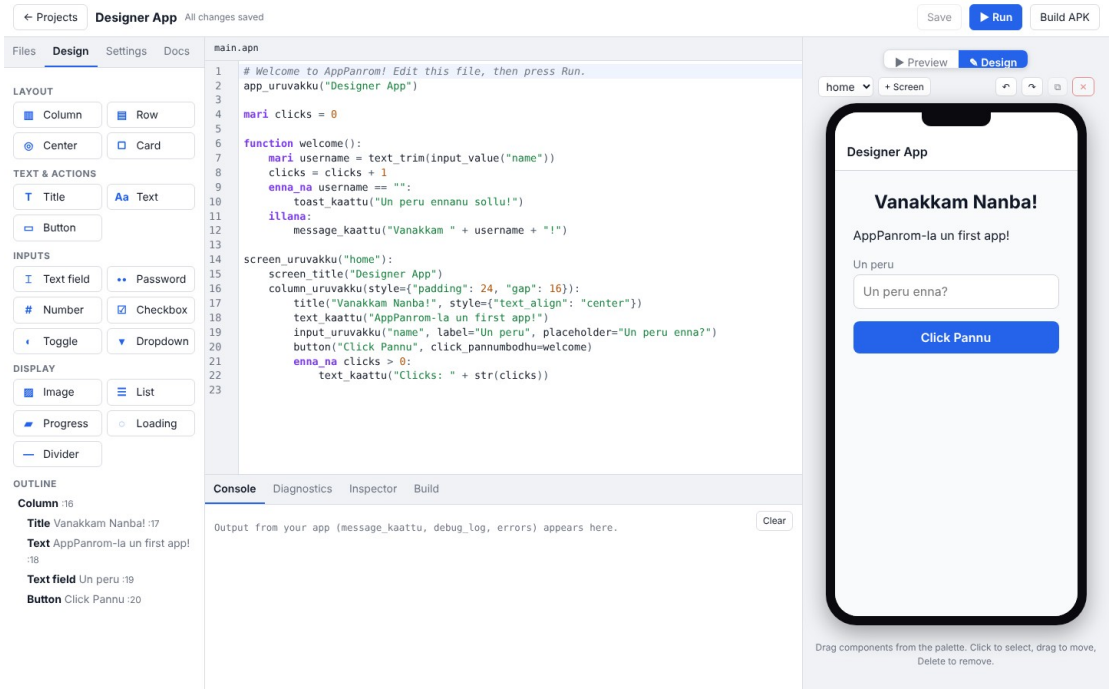


Fig. 5: Visual designer: palette on the left, code in the middle, phone canvas on the right

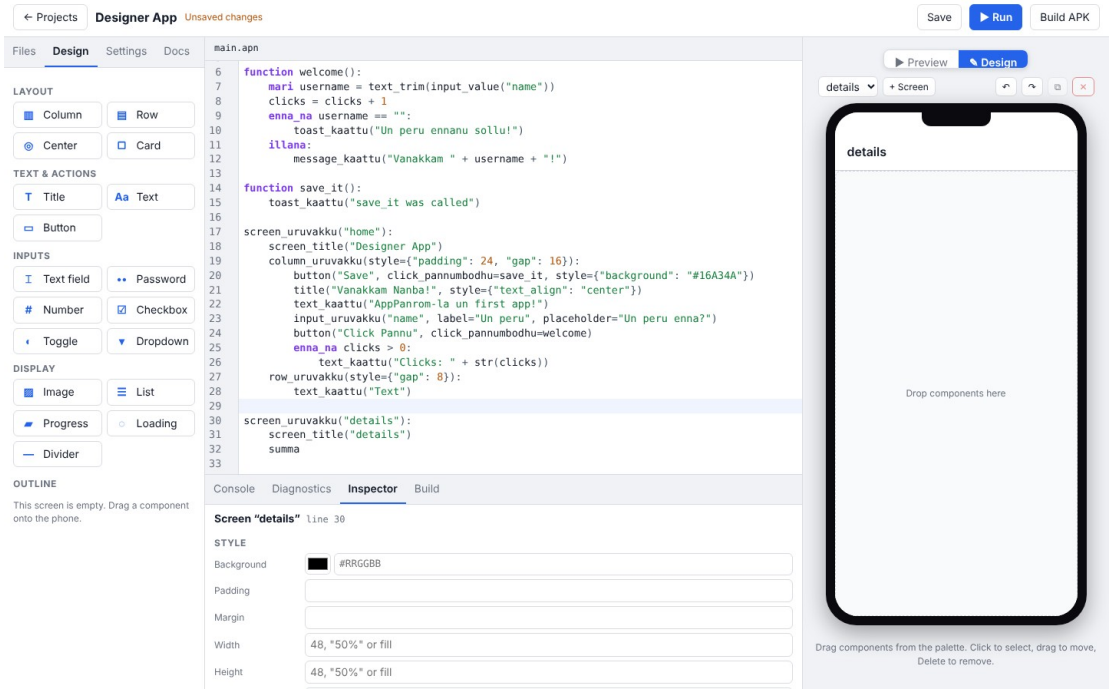


Fig. 6: After dragging components and editing them, the code was updated live; the inspector shows the selected screen

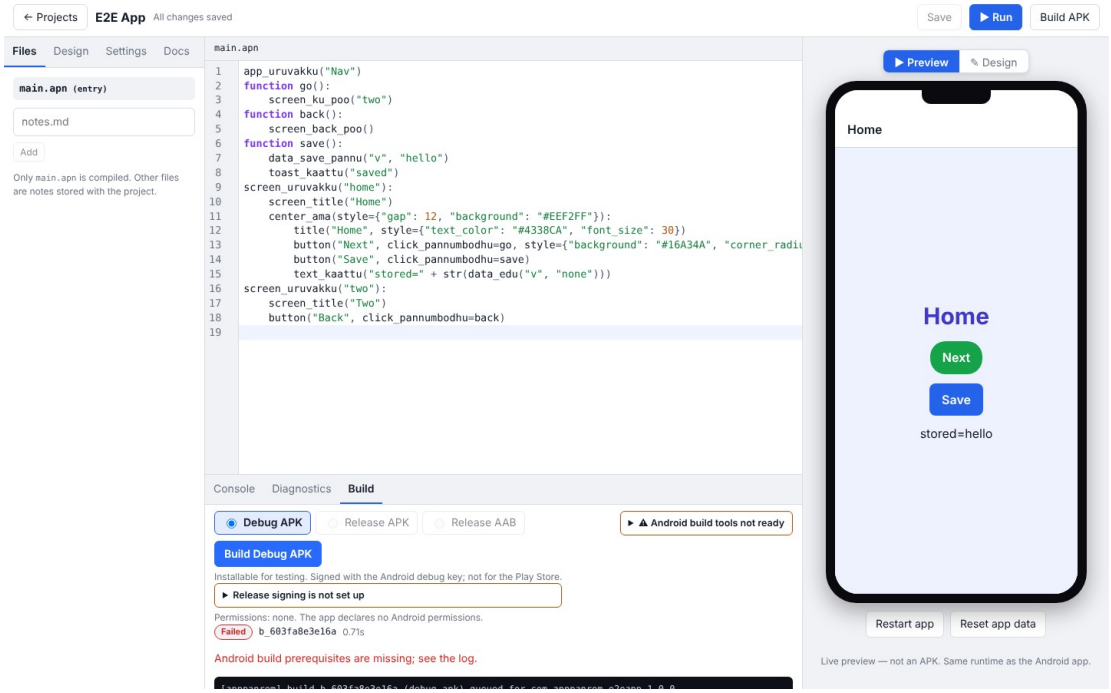


Fig. 7: Build panel: prerequisite check and an honest failed build when the Android SDK is missing

CONCLUSION:

AppPanrom successfully brings a Tanglish programming language, a visual designer and an Android build pipeline together in one studio. Source code passes through lexical, syntax and semantic analysis into a validated intermediate representation that a single shared runtime renders identically in the live preview and in the Android application. The drag-and-drop designer and the code editor edit the same text, so they update each other live. Errors are reported with line numbers and suggestions, resource limits protect against unsafe or endless programs, and the build pipeline only reports success when a verified artifact exists.

Implementation status. The language, runtime, Studio, designer, backend, icon handling, permission declaration and the release-signing and verification logic are implemented and tested automatically (114 backend tests, 30 runtime tests, one frontend unit test and 15 end-to-end scenarios in a real browser). The Android build was exercised with a test double for Gradle together with real keytool and jarsigner, and the generated web part of the APK was run in a browser under the same security policy. A complete Gradle build against the Android SDK, and installation on a device or emulator, could not be performed in the development environment because the SDK download host was not reachable; this is the first item to be done next.

Future work includes verifying real device builds, network access through a validated server-side proxy, runtime-permission features such as camera and location, more themes, and resize handles in the designer.